

# FastAPI Interview Preparation Guide

## From Scratch → Senior (5+ Years Signal)

Step-by-step learning • Practice tasks after every module  
Day-by-day 30-day plan • Portfolio projects • Mock questions

|                 |                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| Who this is for | Engineers targeting FastAPI / Python backend roles at mid-senior+ level who want a structured rebuild from fundamentals |
| Daily time      | 2–3 hours/day for 30 days (or stretch weekends). Consistency > marathon sessions.                                       |
| Outcomes        | Solid mental model, shipped practice APIs, one flagship portfolio project, interview-ready explanations                 |
| Stack           | Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2 async, Postgres, Redis, Docker, pytest                                 |

How to use this book: Read a module → do every checkbox task → only then move on. The 30-day plan maps modules to calendar days. Portfolio projects start in Week 3.

# Table of Contents

---

## Part A — Foundations (start from scratch)

- A0. Setup & mental model of APIs
- A1. Python essentials for FastAPI
- A2. HTTP, REST, and JSON APIs
- A3. Your first FastAPI app

## Part B — Core FastAPI

- B1. Path/query/body, status codes, routers
- B2. Pydantic v2 schemas
- B3. Dependency injection & settings
- B4. Error handling & middleware

## Part C — Data & Auth

- C1. Postgres + SQLAlchemy 2 async
- C2. Alembic migrations & repositories
- C3. AuthN/AuthZ (JWT, roles, tenancy)
- C4. Redis, caching, rate limits

## Part D — Senior depth

- D1. Async concurrency & performance
- D2. Testing & CI
- D3. Observability & production
- D4. Architecture & system design

## Part E — 30-Day Plan (day-by-day)

## Part F — Portfolio projects

## Part G — Question bank & cheat sheets

# A0. Setup & Mental Model of APIs

Before FastAPI itself: you need a clean environment and a clear picture of what an API does. An API is a contract: clients send HTTP requests; your service validates input, applies business rules, reads/writes data, and returns a structured response (usually JSON) with a meaningful status code.

## Install checklist

- Python 3.11+ (verify: `python --version`)
- Git + VS Code/Cursor + Docker Desktop
- Create a practice folder: `fastapi-30day/` with a `venv`
- Install baseline: `pip install fastapi uvicorn[standard] httpx pydantic pydantic-settings`

## Key vocabulary (memorize)

- **Client / Server:** browser, mobile app, or another service talking to your API
- **Endpoint / route:** URL path + HTTP method (`GET /users/1`)
- **Request / Response:** headers + body in; status + headers + body out
- **Stateless:** each request carries what the server needs (often a token)
- **OpenAPI:** machine-readable description of your API (FastAPI generates it)

### PRACTICE TASKS — A0 Setup

- Create repo `fastapi-30day`, `venv`, `.gitignore` (`venv`, `.env`, `__pycache__`)
  - Install packages listed above; run `uvicorn --help` successfully
  - Write `NOTES.md` with your own 5-sentence explanation of request → response
  - Bookmark FastAPI docs + Pydantic v2 docs; skim the tutorial TOC only (don't deep-dive yet)
- Done when:** Repo exists, `venv` works, `NOTES.md` has your mental model.

# A1. Python Essentials for FastAPI

You do not need to relearn all of Python—but FastAPI code leans hard on typing, `async`, context managers, and clean functions. Close these gaps before writing production-looking routes.

## Must be comfortable with

- Type hints: `str` | `None`, `list[User]`, `dict[str, Any]`
- Functions returning typed values; dataclasses / simple classes
- `async def` / `await` basics (I/O wait points)
- Context managers (`with` / `async with`)
- Exceptions: raise custom errors; `try/except/finally`
- Modules/packages: `__init__.py`, imports, avoiding circular imports
- `Virtualenv` + `requirements.txt` or `pyproject.toml`

## Mini async mental model

Async is not 'faster CPU'. It lets one process handle many waiting I/O operations (DB, HTTP, Redis) without blocking. If you run heavy CPU or sync I/O inside `async` code without care, you freeze everyone sharing that event loop.

```
# sync vs async intuition
import asyncio, time

async def fetch_fake(i: int) -> str:
    await asyncio.sleep(0.2)      # non-blocking wait
    return f"ok-{i}"

async def main():
```

```
# runs roughly concurrently → ~0.2s total, not 1.0s
results = await asyncio.gather(*(fetch_fake(i) for i in range(5)))
print(results)
```

```
asyncio.run(main())
```

### PRACTICE TASKS — A1 Python

- Write scripts/typing\_drill.py with 5 typed functions (parse email, clamp int, merge dicts, filter list, safe divide)
  - Write scripts/async\_drill.py using asyncio.gather for 5 fake 'API calls' with sleep
  - Explain in NOTES.md (in your words) when async helps and when it does not
  - Enable a type checker (pyright/mypy) on the drills; fix all errors
- Done when:** Both drills run; NOTES.md has async explanation; type checker clean.

## A2. HTTP, REST, and JSON APIs

Interviewers assume HTTP fluency. Be precise about methods, status codes, headers, and REST resource modeling.

### HTTP methods (practical meaning)

- **GET** — read; safe & idempotent; no body side effects
- **POST** — create or trigger action; not idempotent unless you add Idempotency-Key
- **PUT** — replace resource; idempotent
- **PATCH** — partial update; ideally idempotent for same payload
- **DELETE** — remove; idempotent (deleting again → 404 or 204)

### Status codes you must use correctly

- 200 OK, 201 Created, 202 Accepted, 204 No Content
- 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
- 409 Conflict, 422 Unprocessable Entity (validation), 429 Too Many Requests
- 500 Internal Server Error, 503 Service Unavailable

### REST modeling tips

- Nouns for resources: /users, /orders/{id}/items
- Avoid verbs in paths: prefer POST /orders/{id}/cancel only for non-CRUD actions
- Stable error body: {code, message, details, request\_id}
- Pagination, filtering, sorting as query params with allowlists

### PRACTICE TASKS — A2 HTTP/REST

- On paper (or NOTES.md), design REST resources for a 'Bookstore': books, authors, reviews, carts
  - For each resource list methods + status codes for success and 3 failure cases
  - Use curl or httpie against https://httpbin.org (GET/POST) and inspect headers
  - Write a 10-line JSON error envelope standard you will reuse in later modules
- Done when:** Bookstore API sketch + error envelope committed to NOTES.md.

## A3. Your First FastAPI App

Build the smallest useful app and understand the generated OpenAPI docs. This is your 'hello, ownership' moment.

```
# app/main.py
from fastapi import FastAPI

app = FastAPI(title="Day0 API", version="0.1.0")

@app.get("/health")
def health():
    return {"status": "ok"}
```

```
@app.get("/hello/{name}")
def hello(name: str, shout: bool = False) -> dict[str, str]:
    msg = f"Hello, {name}!"
    return {"message": msg.upper() if shout else msg}

# Run: uvicorn app.main:app --reload
# Docs: http://127.0.0.1:8000/docs
```

## What to notice in /docs

- Schemas inferred from type hints
- Try-it-out executes real requests
- OpenAPI JSON at /openapi.json — clients and tests can consume this

### PRACTICE TASKS — A3 First app

- Create package app/ with main.py as above; run with --reload
  - Add POST /echo that accepts JSON {text: str} and returns length + reversed text
  - Screenshot or export openapi.json into docs/openapi-day0.json
  - Break the handler on purpose; observe default 500; note what a client sees
- Done when:** Health + hello + echo work; you can explain /docs to someone else.

## B1. Path / Query / Body, Status Codes, Routers

FastAPI binds request data via function parameters. Master Path, Query, Body, and APIRouter so apps stay modular.

```
from fastapi import APIRouter, Path, Query, status
from pydantic import BaseModel, Field

router = APIRouter(prefix="/items", tags=["items"])

class ItemCreate(BaseModel):
    name: str = Field(min_length=1, max_length=80)
    price: float = Field(gt=0)

@router.post("", status_code=status.HTTP_201_CREATED)
def create_item(payload: ItemCreate) -> ItemCreate:
    return payload

@router.get("/{item_id}")
def get_item(
    item_id: int = Path(ge=1),
    q: str | None = Query(default=None, max_length=50),
):
    return {"item_id": item_id, "q": q}

# main.py: app.include_router(router)
```

### Senior habits early

- One router per domain area (users, items, orders)
- Explicit status\_code on creates/deletes
- Never put business logic deep in the route forever—extract functions soon
- Use tags/summary/description so OpenAPI stays useful

#### PRACTICE TASKS — B1 Routing

- Build in-memory Items API: create, list (search q), get, update PATCH, delete
  - Use APIRouter; return proper 201/204/404
  - Add query params: skip/limit with validation (0≤skip, 1≤limit≤100)
  - Write curl commands in README for every endpoint
- Done when:** Full CRUD works from /docs and curl; 404 on missing ids.

## B2. Pydantic v2 Schemas

Pydantic is the validation backbone. At interview level you separate Create/Update/Read models and know v2 APIs.

- **BaseModel** + Field constraints; extra='forbid' for strict inputs
- **field\_validator** / **model\_validator** for cross-field rules
- **ConfigDict(from\_attributes=True)** when reading ORM objects
- **model\_dump(exclude\_unset=True)** for PATCH partial updates
- Never return secrets (password hashes) in response models

```
from pydantic import BaseModel, ConfigDict, Field, field_validator

class UserCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    email: str
    password: str = Field(min_length=8)
    display_name: str = Field(min_length=1, max_length=80)

    @field_validator("email")
    @classmethod
    def norm_email(cls, v: str) -> str:
        return v.strip().lower()

class UserUpdate(BaseModel):
```

```

model_config = ConfigDict(extra="forbid")
display_name: str | None = None
email: str | None = None

class UserRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    email: str
    display_name: str

```

#### PRACTICE TASKS — B2 Pydantic

- Refactor Items API to ItemCreate / ItemUpdate / ItemRead
  - Reject unknown fields (extra=forbid); add a price currency field with validation
  - Unit-test validators with plain pytest (no FastAPI) for 5 invalid payloads
  - Document in NOTES.md why response models prevent data leaks
- Done when:** Invalid payloads return 422; unit tests pass; README updated.

## B3. Dependency Injection & Settings

Depends() is how FastAPI shares DB sessions, current user, settings, and feature flags. Seniors design dependency graphs intentionally.

```

from functools import lru_cache
from typing import Annotated
from fastapi import Depends
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    app_name: str = "demo"
    api_keys: str = "dev-key"

@lru_cache
def get_settings() -> Settings:
    return Settings()

SettingsDep = Annotated[Settings, Depends(get_settings)]

def require_api_key(settings: SettingsDep, x_api_key: str | None = None):
    if x_api_key != settings.api_keys:
        from fastapi import HTTPException
        raise HTTPException(401, "Invalid API key")
    return True

```

### Lifespan (startup/shutdown)

Use lifespan context managers to open/close connection pools and HTTP clients. Avoid deprecated on\_event startup/shutdown in new code.

#### PRACTICE TASKS — B3 DI & settings

- Add pydantic-settings Settings; load from .env (do not commit .env)
  - Protect write endpoints with X-API-Key dependency
  - Add lifespan that logs 'startup' / 'shutdown' (later replace with real pools)
  - Override the API-key dependency in a test using app.dependency\_overrides
- Done when:** Writes fail without key; test proves override works.

## B4. Error Handling & Middleware

- Raise HTTPException for expected failures; use custom exception handlers for domain errors
- Middleware for request IDs, timing, CORS, trusted hosts
- Never leak stack traces to clients in production
- Map domain NotFoundError → 404, ConflictError → 409 in one place

```

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
import uuid, time

app = FastAPI()

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    rid = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Request-ID"] = rid
    response.headers["X-Process-Time"] = f"{time.perf_counter()-start:.4f}"
    return response

class NotFoundError(Exception): ...

@app.exception_handler(NotFoundError)
async def not_found_handler(request: Request, exc: NotFoundError):
    return JSONResponse(status_code=404, content={"code": "not_found", "message": str(exc)})

```

### PRACTICE TASKS — B4 Errors & middleware

- Add request-id middleware; return id on every response
  - Create domain exceptions + handlers for not\_found and conflict
  - Standardize error JSON to your A2 envelope (include request\_id)
  - Add a test that asserts 404 body shape
- Done when:** All errors share one envelope; request id always present.

## C1. Postgres + SQLAlchemy 2 Async

Replace in-memory dicts with a real database. Prefer SQLAlchemy 2.x async style with AsyncSession.

- Docker Compose service for Postgres; DATABASE\_URL in .env
- create\_async\_engine + async\_sessionmaker
- Mapped models with type annotations; select( ) API
- One session per request via yield dependency; commit in service layer
- Beware: sync Session inside async routes blocks the event loop

```
# deps.py (sketch)
from collections.abc import AsyncIterator
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker, create_async_engine

engine = create_async_engine(settings.database_url, pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db() -> AsyncIterator[AsyncSession]:
    async with SessionLocal() as session:
        yield session
```

### PRACTICE TASKS — C1 Database

- docker compose up -d postgres; connect with DATABASE\_URL (asyncpg)
  - Create Item model + create/list/get/update/delete using AsyncSession
  - Add unique constraint on name; handle integrity error as 409
  - Verify N+1 isn't an issue yet; note lazy-load risk in NOTES.md
- Done when:** CRUD persists across restarts; 409 on duplicate name.

## C2. Alembic Migrations & Repositories

- Alembic autogenerate for schema changes; review migrations before applying
- Expand/contract mindset for zero-downtime (add column nullable → backfill → constrain)
- Repository functions keep SQL out of routers
- Services orchestrate repositories + rules + transactions

### Suggested layout

```
app/
  api/v1/endpoints/items.py
  schemas/item.py
  models/item.py
  repositories/items.py
  services/items.py
  db/session.py
alembic/
```

### PRACTICE TASKS — C2 Migrations & layers

- Init Alembic; create migration for items; upgrade head
  - Add a new column via migration (e.g. description); upgrade/downgrade works
  - Split code into repository + service + endpoint (no SQL in routes)
  - Write integration test against real Postgres (compose) for create+get
- Done when:** Migration chain clean; routes thin; integration test green.

## C3. AuthN / AuthZ (JWT, Roles, Tenancy)

Authentication = who you are. Authorization = what you may do. JWT carries claims; your code still must enforce permissions on every resource (prevent IDOR/BOLA).

- Register/login; password hash with bcrypt/argon2 (never store plaintext)
- Access token short-lived; optional refresh rotation
- Depends chain: `get_current_user` → `require_role('admin')`
- Object-level checks: `resource.owner_id == user.id` (or tenant membership)
- CORS, secure cookies vs Bearer headers — know trade-offs for SPAs

```
# conceptual auth dependency
async def get_current_user(token: str = Depends(oauth2_scheme), db=Depends(get_db)):
    payload = decode_jwt(token) # raises 401 on failure
    user = await users_repo.get(db, payload["sub"])
    if not user or not user.is_active:
        raise HTTPException(401, "Inactive")
    return user

def require_roles(*roles: str):
    async def _inner(user=Depends(get_current_user)):
        if user.role not in roles:
            raise HTTPException(403, "Forbidden")
        return user
    return _inner
```

### PRACTICE TASKS — C3 Auth

- Add User model; register + login endpoints; return JWT access token
  - Protect item writes: only authenticated users; store `owner_id`
  - Enforce owner-only update/delete (403/404 strategy — pick and document)
  - Tests: user A cannot modify user B's item (IDOR test must pass)
- Done when:** IDOR test fails if you remove the owner check; document auth flow.

## C4. Redis, Caching, Rate Limits

- Cache-aside for hot reads; include tenant/user in keys when needed
- TTL + invalidation on writes; avoid caching unauthorized responses
- Rate limit login and public GETs (token bucket / fixed window in Redis)
- Idempotency-Key storage for unsafe POSTs

### PRACTICE TASKS — C4 Redis

- Add Redis to docker compose; ping from lifespan
  - Cache GET `/items/{id}` for 30s; invalidate on update/delete
  - Rate-limit login to 5/min/IP; return 429 with `Retry-After`
  - Measure before/after cache with a tiny script (10 sequential gets)
- Done when:** Cache hit path works; login limiter returns 429.

## D1. Async Concurrency & Performance

- Reuse `httpx.AsyncClient` / engine pools via lifespan
- `asyncio.gather` / `TaskGroup` for independent I/O; Semaphore to limit fan-out
- Move CPU-bound work to process pool or separate worker
- Find N+1 queries; use `selectinload/joinedload`
- Profile before optimizing (py-spy, DB EXPLAIN, OpenTelemetry)

### PRACTICE TASKS — D1 Performance

- Add an endpoint that fans out 5 outbound HTTP calls with `httpx` + `gather`
- Intentionally block with `time.sleep` in an async route; observe latency impact; fix it
- Add a notes section: top 5 FastAPI performance footguns
- Run a quick load check (hey/k6/locust) on GET list; record p95

**Done when:** You can explain a before/after latency story with numbers.

## D2. Testing & CI

- Unit tests for domain/services with fakes
- API tests: `httpx.AsyncClient` + `ASGITransport`
- Integration: `testcontainers` or `compose` Postgres
- `dependency_overrides` for auth/db
- GitHub Actions: lint + test on PR

```
import pytest
from httpx import ASGITransport, AsyncClient
from app.main import app

@pytest.mark.asyncio
async def test_health():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as ac:
        r = await ac.get("/health")
        assert r.status_code == 200
        assert r.json()["status"] == "ok"
```

### PRACTICE TASKS — D2 Testing

- Achieve API tests for auth happy path + IDOR + validation 422
- Add `ruff` + `pytest` to CI workflow
- Fail CI if tests fail; badge optional
- Write TESTING.md describing how to run tests locally

**Done when:** CI green on a clean clone path documented.

## D3. Observability & Production Hardening

- Structured JSON logs with `request_id`, `user_id`, `tenant_id`
- `/health/live` vs `/health/ready` (DB/Redis checks)
- OpenTelemetry traces (at least conceptual wiring)
- Docker non-root; timeouts on all external calls; graceful shutdown
- Secrets via env; 12-factor config

### PRACTICE TASKS — D3 Production

- Split live/ready probes; ready fails if DB down
  - Dockerfile + compose for api+db+redis; app runs with one command
  - Add timeouts to DB engine and httpx client
  - Write RUNBOOK.md: common failures and first commands to run
- Done when:** compose up works; ready probe documented; runbook exists.

## D4. Architecture & System Design

---

For 5+ interviews: clarify requirements → API → data → async flows → failures → scale → security → ops.

- Modular monolith vs microservices — decide with team size & coupling
- Outbox pattern for reliable events; idempotent consumers
- Idempotency keys for payment-like POSTs
- Background workers (ARQ/Celery) vs BackgroundTasks
- Versioning, pagination, multi-tenant isolation as first-class design

### PRACTICE TASKS — D4 Design

- Write a 1-page ADR: 'Why modular monolith for portfolio Project A'
  - Whiteboard (on paper) a URL shortener: API, data, cache, abuse, scale
  - Whiteboard a notifications system with WS + Redis pub/sub
  - Practice a 10-minute verbal walkthrough; record yourself once
- Done when:** Two design sketches + one recorded walkthrough in your notes.

## How the plan works

Budget ~2–3 hours/day. Each day has Learn + Do. If you miss a day, skip ahead only if the previous day's 'Do' checkboxes are done—skills compound. Weekends can catch up or deepen.

*Weekly themes: W1 Foundations+Core API · W2 Data+Auth · W3 Hardening+Portfolio start · W4 Portfolio polish+Interview drills*

### Week 1 — Foundations & Core FastAPI (Days 1–7)

#### Day 1 — Environment & API mental model

**Learn:** Module A0

**Do:** Create fastapi-30day repo, venv, NOTES.md; install baseline packages; write mental model

#### Day 2 — Python typing & async drills

**Learn:** Module A1

**Do:** Finish typing\_drill.py + async\_drill.py; enable pyright/mypy; update NOTES

#### Day 3 — HTTP/REST design

**Learn:** Module A2

**Do:** Bookstore resource design; error envelope; httpbin curl practice

#### Day 4 — First FastAPI app

**Learn:** Module A3

**Do:** health/hello/echo; save openapi.json; break & observe 500

#### Day 5 — Routing & CRUD in-memory

**Learn:** Module B1

**Do:** Items router CRUD with skip/limit; README curl section

#### Day 6 — Pydantic v2 schemas

**Learn:** Module B2

**Do:** Create/Update/Read models; forbid extras; 5 validator unit tests

#### Day 7 — DI, settings, weekly review

**Learn:** Module B3 (start)

**Do:** Settings+.env; API key on writes; dependency\_overrides test; review Week 1 gaps

### Week 2 — Errors, Database, Auth (Days 8–14)

#### Day 8 — Errors & middleware

**Learn:** Module B4

**Do:** Request-id middleware; domain exception handlers; 404 shape test

#### Day 9 — Postgres + SQLAlchemy async

**Learn:** Module C1

**Do:** Compose Postgres; persist Items; unique → 409

#### Day 10 — Alembic + layered architecture

**Learn:** Module C2

**Do:** Migrations; repository/service split; integration test

#### Day 11 — Users & password hashing

**Learn:** Module C3 (part 1)

**Do:** User model; register; hash passwords; never return hash

#### Day 12 — JWT login & protected routes

**Learn:** Module C3 (part 2)

**Do:** Login issues JWT; protect writes; store owner\_id

### **Day 13 — Authorization / IDOR defense**

**Learn:** Module C3 (part 3)

**Do:** Owner-only update/delete; mandatory IDOR tests

### **Day 14 — Redis cache + rate limit**

**Learn:** Module C4

**Do:** Cache get-by-id; invalidate on write; login 429 limiter; Week 2 retro

## Week 3 — Senior topics & Portfolio kickoff (Days 15–21)

### Day 15 — Async performance

**Learn:** Module D1

**Do:** httpx gather endpoint; fix sleep footgun; capture p95 notes

### Day 16 — Testing strategy & CI

**Learn:** Module D2

**Do:** Expand API tests; ruff+pytest GitHub Action; TESTING.md

### Day 17 — Observability & Docker

**Learn:** Module D3

**Do:** live/ready probes; Dockerfile; one-command compose; RUNBOOK.md

### Day 18 — System design practice

**Learn:** Module D4

**Do:** URL shortener + notifications sketches; record 10-min walkthrough

### Day 19 — Portfolio Project A kickoff

**Learn:** Part F — Project A

**Do:** Scaffold multi-tenant billing API skeleton; choose modules; first migration

### Day 20 — Project A — tenancy & RBAC

**Learn:** Project A features

**Do:** Org/members/roles; tenant scoping in repositories; tests for isolation

### Day 21 — Project A — billing core

**Learn:** Project A features

**Do:** Plans + subscriptions models; create subscription endpoint; Week 3 demo to a friend/rubber duck

## Week 4 — Ship, polish, interview mode (Days 22–30)

### Day 22 — Idempotent usage ingest

**Learn:** Project A

**Do:** POST /usage with Idempotency-Key + Redis/DB store; retry tests

### Day 23 — Workers & invoices

**Learn:** Project A

**Do:** Background worker generates invoice stub; 202 job pattern or outbox notes

### Day 24 — Security pass

**Learn:** Project A + OWASP

**Do:** Threat notes in README (BOLA, webhook forgery); fix gaps; rate limits

### Day 25 — Project B lite (optional) or deepen A

**Learn:** Notifications lite OR A polish

**Do:** Either WS+Redis pub/sub mini demo OR improve A's OpenAPI examples & coverage

### Day 26 — Docs & architecture narrative

**Learn:** Portfolio polish

**Do:** ARCHITECTURE.md + ADR + Postman/Bruno collection; demo GIF/Loom

### Day 27 — Mock interview — coding

**Learn:** Question bank Part G

**Do:** 90 min: implement a small feature on a timer; narrate trade-offs

### Day 28 — Mock interview — system design

**Learn:** Part G design prompts

**Do:** 45 min design + 15 min critique against checklist (failures, scale, auth)

### Day 29 — Behavioral stories + resume bullets

**Learn:** Leadership framing

**Do:** Write 5 STAR stories; turn Project A into 3 metric-style resume bullets

### Day 30 — Final dry run & gap list

**Learn:** Everything

**Do:** Full review: redo weakest Day tasks; list remaining gaps; plan next 2 weeks

## 30-day success checklist

- You can build a FastAPI service from zero with settings, routers, schemas, DB, auth
- You have automated tests + CI + Docker Compose
- You can explain async pitfalls, IDOR, idempotency, and ready vs live probes
- Portfolio Project A is demoable with README architecture and security notes
- You completed at least one timed coding mock and one system-design mock

***If you only have 1.5 hours/day:*** keep Learn shorter, never skip Do tasks on Days 4–14 (core skill days). Stretch Days 19–26 over an extra week rather than skipping auth/IDOR.

### F. Portfolio Projects (Interview Signal)

Ship quality over quantity. Finish Project A well. Add B or C if time remains. Each project needs: Compose, tests, OpenAPI, ARCHITECTURE.md, security notes.

#### Project A — Multi-Tenant SaaS Billing API (Flagship)

B2B SaaS backend: organizations, members/RBAC, plans, subscriptions, usage metering, invoices. This is your senior signal project for the 30-day plan (Days 19–26).

##### Build milestones (use as task list)

- M1 Skeleton: app layout, settings, Postgres, Alembic, health probes
- M2 Identity: users, JWT, orgs, memberships, roles
- M3 Catalog: plans; subscribe/cancel with validations
- M4 Usage: ingest events with Idempotency-Key; aggregate
- M5 Billing job: generate invoice stub via worker; list invoices
- M6 Hardening: tenant isolation tests, rate limits, RUNBOOK, Loom demo

##### PRACTICE TASKS — Project A acceptance

- User in Org A cannot read Org B invoices (automated test)
  - Usage POST retries with same Idempotency-Key return identical results
  - OpenAPI documents auth + error envelope
  - ARCHITECTURE.md explains tenancy + job flow + failure modes
- Done when:** Demo works from compose up; you can narrate design in 10 minutes.

#### Project B — Notifications Hub (Async / Realtime)

REST for preferences/history + WebSocket/SSE push + Redis pub/sub across workers. Great for proving async and horizontal scale awareness.

- Ingest notification events; persist history with cursor pagination
- WS auth on connect; fan-out via Redis; heartbeat
- Delivery retries + DLQ sketch; quiet hours
- k6/hey script for concurrent connections

##### PRACTICE TASKS — Project B acceptance

- Two API replicas both receive pub/sub and push to their clients
  - Unauthorized websocket rejected
  - Failure mode notes: Redis blip behavior
- Done when:** Diagram + working local demo.

#### Project C — Secure Document Processing API

Presigned upload to MinIO/S3, async virus-scan/metadata worker, ACL downloads. Highlights trust boundaries and streaming/object storage.

##### PRACTICE TASKS — Project C acceptance

- Upload initiation → S3 PUT → complete callback flow works in Compose
- ACL enforced on download; expired share links fail
- Worker retries are idempotent for the same object key

**Done when:** Security section in README covers SSRF and ACL.

## Repo polish checklist (all projects)

- Makefile: up, test, lint, migrate
- Conventional commits; clean README quickstart
- Bruno/Postman collection; example .env.example
- CI on PR; coverage for services at least
- 3-minute Loom: problem → architecture → scariest risk → next step at 10× load

## G1. Progressive question bank

### Beginner → intermediate (after Week 1)

**Q1. What does FastAPI generate automatically from your code?**

A: OpenAPI schema + interactive docs from types, paths, and models.

**Q2. Difference between path params, query params, and body?**

A: Path = resource identity; query = filters/options; body = payload for write operations.

**Q3. Why use response\_model / Read schemas?**

A: Control output fields, validate outbound data, prevent leaking internals.

**Q4. What is Depends used for?**

A: Reusable request-scoped wiring: DB, auth, settings, permissions.

### Core senior (after Week 2–3)

**Q1. def vs async def endpoints — when each?**

A: async for native async I/O; def runs in threadpool for sync libs; never block the loop with sync I/O/CPU.

**Q2. How do you prevent IDOR in multi-tenant APIs?**

A: Authorize on every fetch with tenant/owner scope; deny-by-default; automated isolation tests.

**Q3. Pydantic v2 vs v1 highlights?**

A: ConfigDict, new validators API, from\_attributes, faster validation, model\_dump changes.

**Q4. BackgroundTasks vs Celery/ARQ?**

A: BackgroundTasks = in-process after response; queues = durable, multi-worker, retries, DLQ.

**Q5. live vs ready probes?**

A: live = process up; ready = dependencies healthy enough to take traffic.

**Q6. How do you design idempotent POSTs?**

A: Idempotency-Key stored with request hash/response; retries return first result.

### System design prompts (Week 4)

- Design a rate limiter used by many FastAPI services
- Design idempotent payments capture
- Migrate Flask monolith /api to FastAPI without downtime
- Design audit logging that is queryable and tamper-evident
- Design multi-tenant feature flags with near-realtime updates

### Debugging scenarios

- p99 latency spike after deploy — investigation steps?
- Intermittent 401s after scaling replicas — causes?
- Async workers exhaust DB connections — what went wrong?
- WebSocket clients miss messages on pod restart — fix?

## G2. Cheat sheets

### Status codes quick map

- 200/201/202/204 success family
- 400/401/403/404/409/422/429 client & conflict family
- 500/503 server & dependency family

### Annotated deps pattern

```
UserDep = Annotated[User, Depends(get_current_user)]
DbDep = Annotated[AsyncSession, Depends(get_db)]

@router.get("/me")
async def me(user: UserDep) -> UserRead:
    return user
```

## Interview phrase bank

- blast radius · rollback plan · explicit consistency model
- measure p99 · backpressure · deny by default
- expand/contract migration · idempotent consumer · outbox

## G3. Behavioral (5+ years)

---

- Incident story: detect → mitigate → root cause → prevent
  - Disagreement on microservices split — align on metrics
  - Mentorship: review checklist / ADR template impact
  - Cut scope to hit a date without sacrificing security
  - Tech debt paid with clear ROI
- 

## Closing coaching

You are not studying to recite FastAPI docs. You are training to own a service: correctness, security, operability, and clear trade-offs. Follow modules in order, tick every practice task, and execute the 30-day plan. On Day 30, your weakest topics become your next two-week focus list—then book real mocks.

*Start Day 1 today: create the repo, write your API mental model in NOTES.md, and do not skip the checkbox tasks. Momentum beats perfect plans.*